

Desafio Técnico — API de Gestão de Pedidos

Contexto

O seu desafio é desenvolver uma API REST em .NET para gerenciamento de clientes, produtos, estoque e pedidos.

A solução deve permitir o cadastro de clientes e produtos, a criação de pedidos, o controle de estoque, o cálculo automático de valores, a alteração controlada do status dos pedidos e o registro do histórico dessas alterações.

A entrega deve demonstrar boas práticas de desenvolvimento backend, organização de código, clareza arquitetural, aplicação de regras de negócio, persistência de dados, validações, testes automatizados e documentação técnica.

Objetivo

Desenvolver uma API em .NET para gerenciamento de pedidos, contemplando:

- cadastro e consulta de clientes;
- cadastro, consulta e atualização de produtos;
- controle de estoque de produtos;
- criação e consulta de pedidos;
- baixa automática de estoque ao criar pedidos;
- retorno de estoque em cancelamentos permitidos;
- alteração de status dos pedidos;
- registro de histórico/auditoria das alterações de status;
- persistência dos dados em banco relacional;
- validações de entrada;
- documentação da API;
- testes automatizados das principais regras de negócio.

Requisitos funcionais

1. Clientes

A API deve permitir:

- cadastrar um cliente;

- consultar um cliente por ID;
- listar clientes cadastrados;
- desativar um cliente.

Dados mínimos do cliente:

- Id
- Nome
- E-mail
- Documento (pode ser CPF/CNPJ ou apenas um identificador único)
- Ativo/Inativo;
- Data de criação;
- Data de atualização.

Regras:

- O nome do cliente é obrigatório.
- O e-mail é obrigatório e deve possuir formato válido.
- O documento é obrigatório.
- Não deve ser permitido cadastrar dois clientes ativos com o mesmo e-mail.
- Não deve ser permitido cadastrar dois clientes ativos com o mesmo documento.
- Caso o documento seja CPF ou CNPJ, a validação deve ser feita algoritmicamente, não apenas por máscara ou quantidade de caracteres.
- Clientes inativos não podem criar pedidos.
- Clientes inativos devem continuar disponíveis para consulta histórica.

2. Produtos

A API deve permitir:

- cadastrar um produto;
- consultar um produto por ID;
- listar produtos cadastrados;
- atualizar dados do produto;
- atualizar preço do produto;
- atualizar estoque do produto;
- ativar ou inativar um produto.

Dados mínimos do produto:

- Id;
- Nome;
- Descrição;
- Preço;
- Estoque disponível;
- Ativo/Inativo;
- Data de criação;
- Data de atualização.

Regras:

- O nome do produto é obrigatório.
- O preço deve ser maior que zero.
- O estoque não pode ser negativo.
- Produtos inativos não podem ser adicionados a novos pedidos.
- Produtos inativos não devem sofrer movimentação de estoque em novos pedidos.
- Produtos inativos devem continuar disponíveis para consulta histórica.
- Produtos já vinculados a pedidos não devem ser removidos fisicamente da base.
- A alteração de preço do produto não deve alterar pedidos já criados.
- O preço do item do pedido deve refletir o preço do produto no momento da criação do pedido.
- A quantidade em estoque deve ser validada no momento da criação do pedido.
- Não deve ser possível criar pedido com produto sem estoque suficiente.

3. Pedidos

A API deve permitir:

- criar um pedido para um cliente;
- adicionar um ou mais itens ao pedido no momento da criação;
- consultar pedido por ID;
- listar pedidos;
- alterar status do pedido.

Dados mínimos do pedido:

- Id;
- ClientId;
- Data de criação;
- Status;
- Valor total;
- Lista de itens.

Dados mínimos do item do pedido:

- Id;
- PedidoId;
- ProdutoId;
- Quantidade;
- Preço unitário no momento da compra;
- Valor total do item.

Regras

- Um pedido deve estar associado a um cliente existente.
- O cliente deve estar ativo no momento da criação do pedido.
- O pedido deve possuir ao menos um item.
- A quantidade de cada item deve ser maior que zero.
- Cada produto informado deve existir.
- Cada produto informado deve estar ativo no momento da criação do pedido.
- Cada produto informado deve possuir estoque suficiente.
- O preço unitário do item deve ser obtido a partir do cadastro do produto no momento da criação do pedido.
- O valor total do item deve ser calculado pela aplicação.
- O valor total do pedido deve ser calculado pela soma dos itens.
- A API não deve aceitar o valor total do pedido como fonte confiável enviada pelo cliente da API.
- A API não deve aceitar o preço unitário do item como fonte confiável enviada pelo cliente da API.
- Ao criar um pedido, o estoque dos produtos deve ser debitado.
- A criação do pedido e a baixa de estoque devem ocorrer de forma consistente.
- Após criado, o pedido deve manter o histórico dos preços praticados, mesmo que o preço do produto seja alterado futuramente.

4. Controle de estoque

O estoque deve representar a quantidade disponível de cada produto para novos pedidos.

Regras

- O estoque inicial do produto deve ser informado no cadastro ou atualizado posteriormente.
- O estoque não pode ser negativo.
- Não deve ser possível criar pedido com quantidade superior ao estoque disponível.
- Ao criar um pedido, a quantidade comprada deve ser debitada do estoque do produto.
- Caso o pedido contenha mais de um item, a baixa de estoque deve considerar todos os itens.
- Caso qualquer item do pedido seja inválido, o pedido não deve ser criado e nenhum estoque deve ser debitado parcialmente.
- A baixa de estoque deve ser transacional.
- Produtos inativos não podem ter estoque debitado em novos pedidos.
- O cancelamento de um pedido deve retornar o estoque dos itens, desde que o pedido ainda não tenha sido enviado.
- Pedido enviado não deve retornar estoque.

5. Fluxo de status do pedido

O pedido deve ter os seguintes status:

- Criado
- Pago
- Enviado
- Cancelado

Regras de transição - Permitir apenas as seguintes transições:

- Criado → Pago
- Pago → Enviado
- Criado → Cancelado

Regras adicionais

- Um pedido enviado não pode ser cancelado.
- Um pedido cancelado não pode ter seu status alterado.
- Um pedido pago não pode voltar para criado.
- Ao cancelar um pedido, o estoque dos itens deve ser retornado apenas se o pedido ainda não tiver sido enviado.
- Toda transição de status deve ser registrada em histórico/auditoria.
- O motivo da alteração de status pode ser opcional.
- Caso o status enviado seja igual ao status atual, o comportamento deve ser tratado de forma consistente e documentado.

6. Histórico de status do pedido

Dados mínimos do histórico

- Id;
- Pedidoid;
- Status anterior;
- Novo status;
- Data/hora da alteração;
- Motivo opcional.

Regras

- O primeiro status do pedido deve ser "criado".
- A criação do pedido pode ou não gerar um registro inicial no histórico, desde que a decisão seja documentada.
- Alterações inválidas de status não devem gerar registro de histórico.
- O histórico deve ser retornado na consulta detalhada do pedido ou em endpoint específico, conforme decisão de implementação.

Requisitos não funcionais

- Utilizar .NET 8 ou 10

- Expor uma API REST
- Persistir os dados em banco relacional
- Utilizar Entity Framework Core ou justificar outra abordagem
- Implementar validações de entrada
- Retornar códigos HTTP adequados
- Organizar o projeto de forma clara e coesa
- Valores monetários: justificar no README o tipo escolhido e a estratégia de arredondamento
- Timestamps devem ser persistidos em UTC; a API deve receber e devolver datas considerando o fuso America/Sao_Paulo
- Descrever no README como a solução se comporta sob requisições simultâneas que disputam o mesmo estoque
- O enunciado pode conter pontos abertos ou aparentemente conflitantes; documente sua interpretação e justificativa no README
- Disponibilizar documentação interativa da API via Swagger/OpenAPI, com schemas de requisição e resposta corretamente tipados, descrição de respostas de erro e exemplos quando aplicável
- Endpoints de listagem devem suportar paginação; descrever no README a estratégia adotada (parâmetros, formato da resposta e como o cliente paginar resultados)
- Implementar testes automatizados cobrindo as principais regras de negócio. Justificar no README os tipos de teste priorizados, a estratégia de cobertura e as ferramentas utilizadas

Endpoints esperados (Não obrigatório)

Clientes

- POST /api/clientes
- GET /api/clientes
- GET /api/clientes/{id}
- PATCH /api/clientes/{id}/status

Produtos

- POST /api/produtos
- GET /api/produtos
- GET /api/produtos/{id}
- PUT /api/produtos/{id}
- PATCH /api/produtos/{id}/status
- PATCH /api/produtos/{id}/estoque

Pedidos

- POST /api/pedidos
- GET /api/pedidos
- GET /api/pedidos/{id}
- PATCH /api/pedidos/{id}/status

Entregáveis e avaliação

Além do código, o repositório deve conter, no mínimo os seguintes:

- **README e documentação técnica**
 - Instruções claras para executar a aplicação localmente.
 - Estratégia de testes adotada (tipos, escopo de cobertura e porque foi escolhido da seguinte forma) e como executar.
 - Descrição das tecnologias utilizadas.
 - Explicação da estrutura da solução.
 - Registro das principais decisões técnicas e trade-offs.
 - Justificativa da estratégia de validação.
 - Justificativa da estratégia de persistência.
 - Explicação da estratégia de estoque.
 - Explicação sobre valores monetários e arredondamento.
 - Explicação sobre datas, UTC e fuso horário.
 - Indicação dos pontos que ficaram fora do escopo e como você abordaria caso tivesse mais tempo.
- **Histórico de commits**
 - Commits incrementais e coerentes com a evolução da solução.
 - Mensagens de commit objetivas e descritivas.
 - Ausência de concentração de toda a entrega em um único commit final.